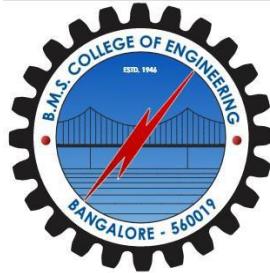


**B.M.S. COLLEGE OF ENGINEERING**  
(Autonomous Institute, Affiliated to VTU)



**Course – Mobile Application Development**  
Course Code – 22IS5PCMAD

**Topic: Productivity Playbook**

**Submitted By**  
Shreya B (1BM21IS166)  
Thanuja M R(1BM21IS194)  
Vaishnavi Basappa Huvannavar (1BM21IS198)

**Under the guidance of**  
Rashmi R  
Assistant Professor

**DEPARTMENT OF INFORMATION SCIENCE & ENGINEERING**

**AY 2023-2024**

## TABLE OF CONTENTS

| Topics                               | Page No. |
|--------------------------------------|----------|
| Abstract                             | 1        |
| Introduction                         | 2        |
| Requirement Specification            | 3-4      |
| System Architecture                  | 5-7      |
| User interface design and Navigation | 8        |
| Implementation details               | 9-17     |
| Results                              | 18-35    |
| Conclusion and further Enhancement   | 36       |
| References                           | 37       |

## **ABSTRACT:**

The Task Reminder Application is designed with user-friendliness and intuitive operation as its core principles, catering to the diverse needs and preferences of its users. Its features are meticulously crafted to offer a seamless experience in task organization and prioritization. Users can input tasks and schedule them according to their preferences, whether it's a one-time event, a recurring task, or a long-term project, providing flexibility in deadline setting and time allocation. Customizable reminders ensure that important tasks are never overlooked, allowing users to set notifications for upcoming deadlines. Deadline tracking becomes effortless with the application's clear overview of tasks and their due dates, helping users stay on track with their commitments. Moreover, notification functionalities keep users informed of any changes or updates to their tasks, ensuring timely updates on task progress or modifications. By providing these features, the Task Reminder Application aims to enhance productivity and reduce stress levels by offering a centralized platform for task management. By encouraging effective task prioritization, the application enables users to focus on what truly matters and achieve a sense of accomplishment and fulfillment. In today's fast-paced world, maintaining a healthy work-life balance is paramount, and the Task Reminder Application empowers individuals to take control of their time usage, allocating time for work, leisure, and personal growth. Ultimately, the application serves as a valuable tool for optimizing time management skills, achieving goals, and leading fulfilling lives.

## **INTRODUCTION:**

In today's fast-paced society, where time is a precious commodity and multitasking has become the norm, the need for efficient task management has never been more critical. Individuals find themselves juggling various responsibilities, from work assignments to personal commitments, and staying organized amidst the chaos can often feel like an insurmountable challenge. As the demands on our time continue to increase, it's easy to become overwhelmed by the sheer volume of tasks and deadlines we must contend with on a daily basis.

In this context, the development of a Task Reminder Application presents an exciting opportunity to revolutionize the way we manage our tasks and deadlines. By harnessing the power of technology, this application aims to provide users with a comprehensive solution for organizing their schedules, setting reminders, and staying on top of their commitments. Whether it's a crucial work deadline, a family event, or a personal goal, the Task Reminder Application offers users a centralized platform to keep track of all their tasks and ensure nothing falls through the cracks.

But beyond just offering convenience, the Task Reminder Application has the potential to significantly improve users' productivity and overall well-being. By helping individuals prioritize their tasks and allocate their time more effectively, the application can reduce stress and anxiety associated with looming deadlines and forgotten commitments. Additionally, by providing timely reminders and notifications, the application empowers users to stay focused and motivated, ultimately enhancing their ability to achieve their goals and aspirations. In this application we will delve deeper into the importance of task management in today's fast-paced society and explore the myriad benefits that a Task Reminder Application can bring to users' lives. From increased productivity to reduced stress levels, the potential impact of this application extends far beyond simply keeping track of tasks—it has the power to transform the way we manage our time and accomplish our goals.

## REQUIREMENT SPECIFICATION:

- Task Management:
  - Users should be able to create, edit, and delete tasks.
  - Tasks should support attributes such as title, description, due date, priority, and category.
- Reminder System:
  - Users should have the ability to set reminders for individual tasks.
  - Reminders should support various notification methods such as push notifications, email notifications.
- Deadline Tracking:
  - The application should display upcoming task deadlines in a clear and organized manner.
  - Users should receive notifications as deadlines approach to ensure timely completion of tasks.
- User Authentication and Authorization:
  - Secure user authentication and authorization mechanisms should be implemented to protect user data and ensure privacy.
  - Users should have the option to log in using email/password
  - The UI should be intuitive and user-friendly, allowing users to navigate tasks and reminders effortlessly.
- Performance:
  - The application should load quickly and respond promptly to user interactions.
  - Response times for task creation, editing, and deletion should be minimal.
- Security:
  - User data should be encrypted both in transit and at rest to prevent unauthorized access.
  - The application should be able to handle a large number of users and tasks without significant degradation in performance.
- Accessibility:
  - The application should comply with accessibility standards (e.g., WCAG) to ensure it is usable by individuals with disabilities.
  - Features such as screen reader compatibility and keyboard navigation should be implemented to facilitate accessibility.
- Compatibility:
  - The application should be compatible with a wide range of devices and web browsers.

- Compatibility testing should be performed to ensure consistent performance across different platforms.

## SYSTEM ARCHITECTURE:

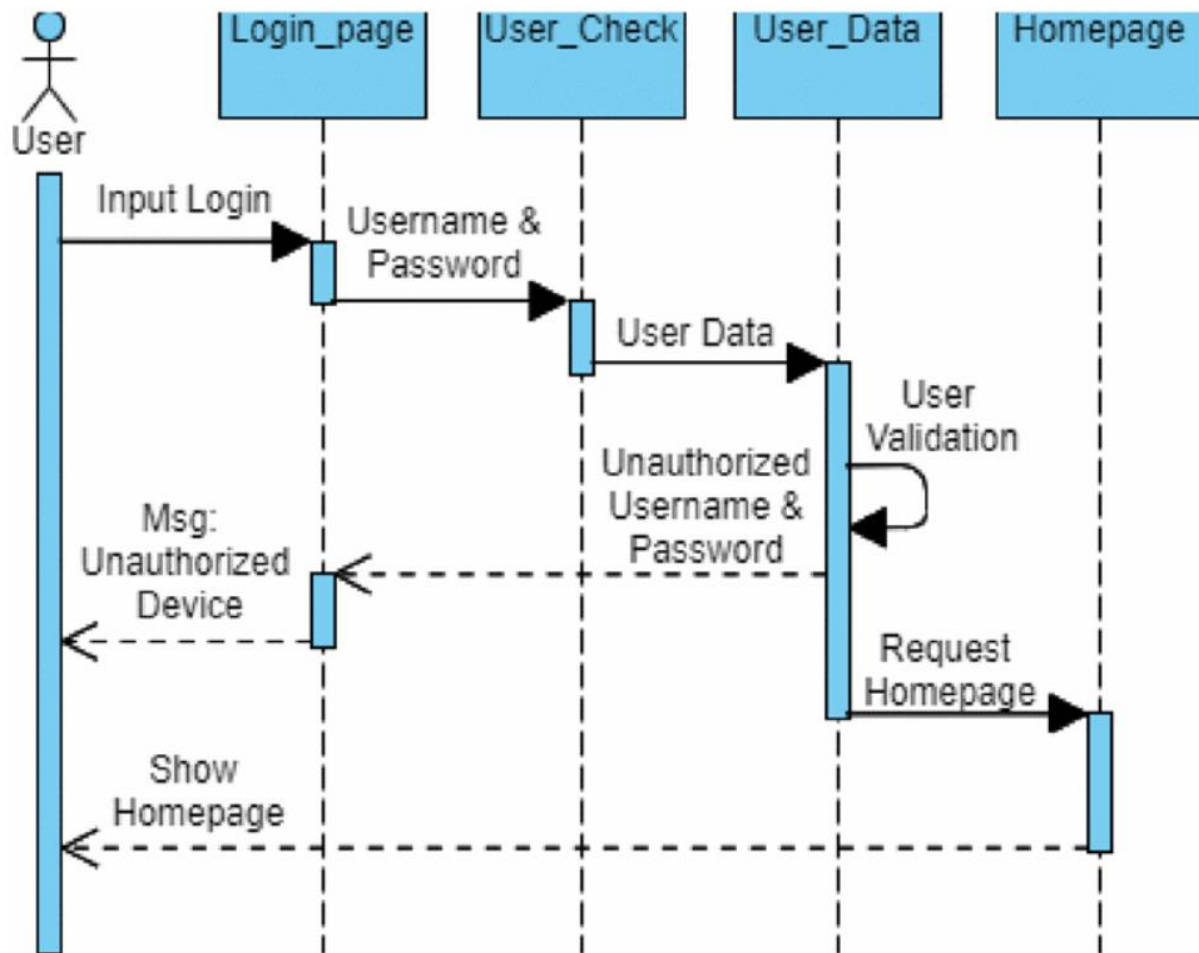
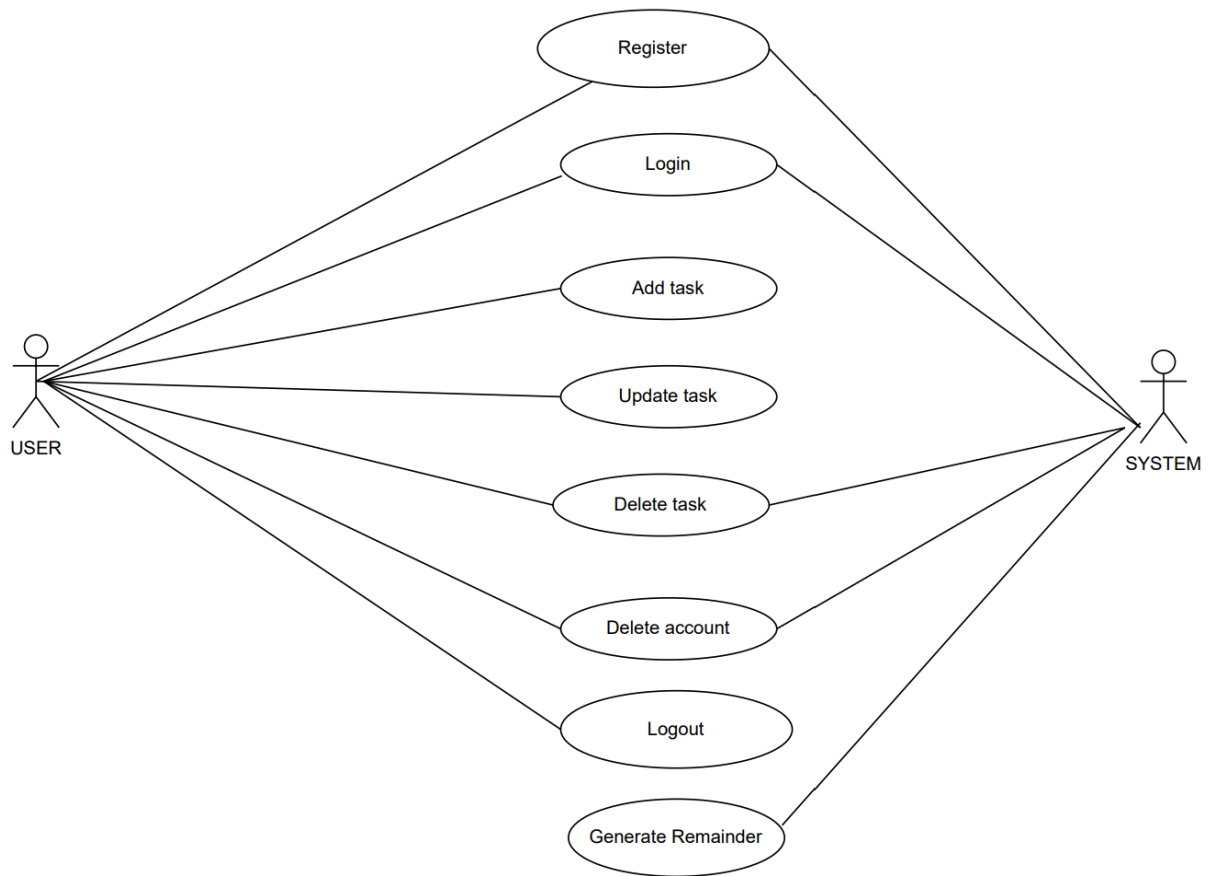
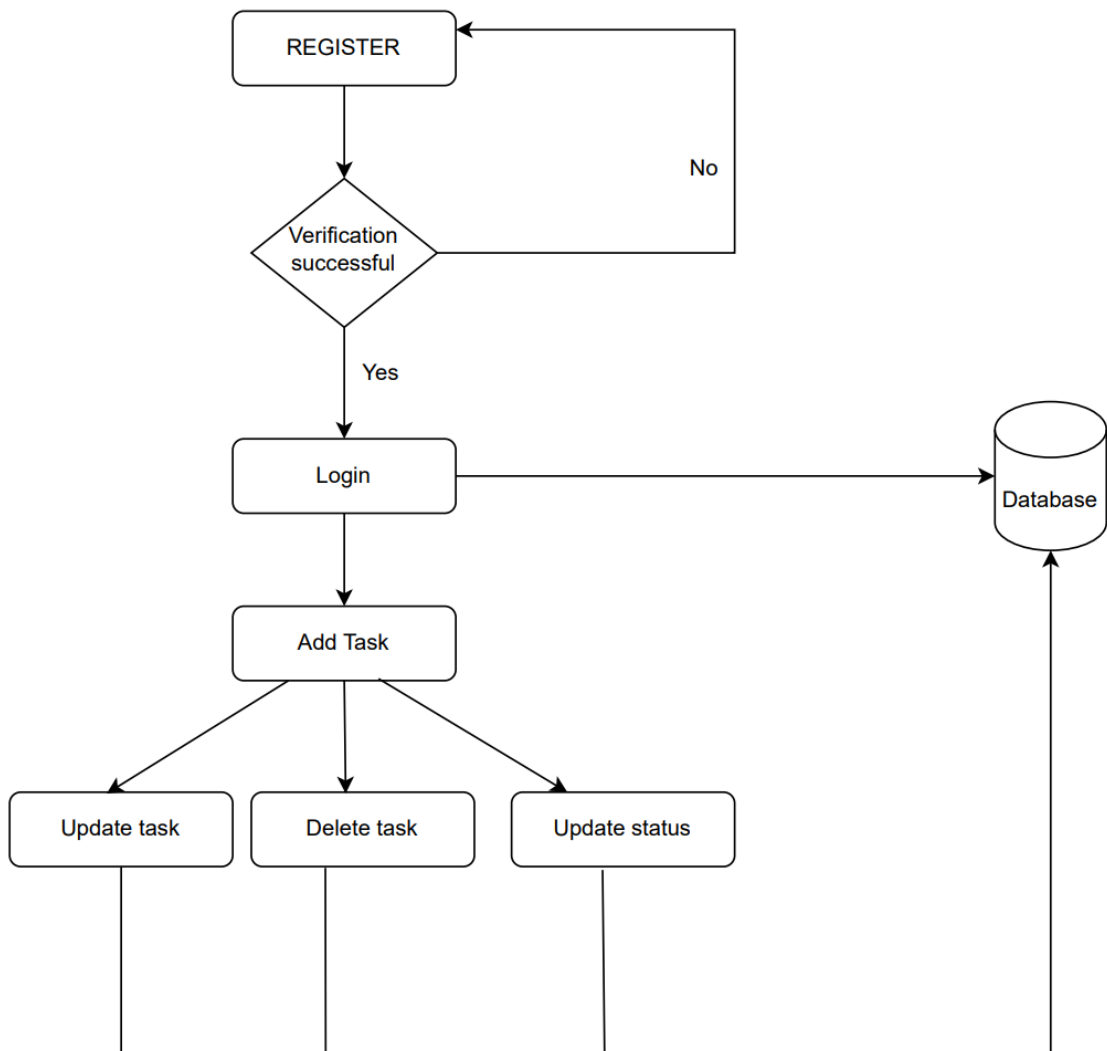


Fig 1. User Authentication Sequence Diagram



**Fig 2. Use Case Diagram**





**Fig 3. Flow Chart**

## **USER INTERFACE DESIGN AND NAVIGATION:**

1. Splash Screen:
  - a. The application displays a splash screen upon launch, providing users with a visually engaging welcome screen while the app loads.
  - b. The splash screen may feature the application logo or branding elements, accompanied by a loading indicator to convey progress.
2. Onboarding Pages:
  - a. Upon first-time installation or after an update, the application presents onboarding pages to guide users through key features and functionalities.
  - b. Onboarding pages typically consist of a series of screens with informative content, including brief descriptions and visuals illustrating how to use the application.
  - c. Users can navigate through onboarding pages using swipe gestures or navigation buttons, with an option to skip or revisit onboarding later.
3. Task Delete and Update:
  - a. Dropdown menu or toolbar, allows users to select delete or update the task.
  - b. Upon on choosing an option the app modifies selected task data (delete or update).
4. Alarm and Notification:
  - a. The application supports alarms and notifications to remind users of upcoming tasks and deadlines.
  - b. Alarms trigger notifications at the specified time, alerting users to important tasks and prompting them to take action.
  - c. Notification includes task title.
5. Authentication:
  - a. The application implements user authentication to ensure secure access to user data and personalized features.
  - b. Authentication mechanisms may include email/password login.
  - c. Users are prompted to authenticate upon launching the application or accessing protected features, with options for new user registration or password recovery if needed.
  - d. Upon successful authentication, users gain access to their personalized task lists, reminders, and other application features, ensuring a seamless and secure user experience.

## IMPLEMENTATION DETAILS: CODE

### 1.View pager fragment:

```
package com.example.to_do_auth.onboarding

import ...

class Viewpagerfragment : Fragment() {

    private var totalPages = 2
    private var currentPage = 0

    override fun onCreateView(
        inflater: LayoutInflater, container: ViewGroup?,
        savedInstanceState: Bundle?
    ): View? {
        // Inflate the layout for this fragment
        val view = inflater.inflate(R.layout.fragment_viewpagerfragment, container, attachToRoot: false)

        val fragmentlist = arrayListOf<Fragment>{
            onboarding1(),
            onboarding2()
        }

        val adapter = Viewpageradapter(
            fragmentlist,
            requireActivity().supportFragmentManager,
            lifecycle
        )

        view.findViewById<ViewPager2>(R.id.viewpager).adapter = adapter
    }
}
```

Viewpageradapter provides fragments for a view pager:

Stores fragment list.

Returns fragment count.

Retrieves fragment for a position.

This manages fragments displayed in an Android app

## 2.View page adaptor:

```
package com.example.to_do_auth.onboarding

import ...

class Viewpageradapter(list:ArrayList<Fragment>, fm: FragmentManager, lifecycle: Lifecycle): FragmentStateAdapter(fm, lifecycle) {

    val fragmentlist=list

    override fun getItemCount(): Int {
        return fragmentlist.size
    }

    override fun createFragment(position: Int): Fragment {
        return fragmentlist[position]
    }

}
```

Onboarding Fragment (Simplified):

Manages :1page onboarding flow.2. Uses View Pager to switch between pages.

Adapter (Viewpageradapter) provides fragments.

Fragment onboarding1 & onboarding2 define content.

This snippet sets up a basic onboarding screen in an Android app.

## 3.Creating account and verifying email using firebase Api:

```
else if(password==confpassword)
{
    auth.createUserWithEmailAndPassword(email, password)
        .addOnCompleteListener(this) { task ->
            if (task.isSuccessful) {
                val user = FirebaseAuth.getCurrentUser()
                var documentreference:DocumentReference
                user!!.sendEmailVerification()
                .addOnCompleteListener { task ->
                    if (task.isSuccessful) {
                        var userID = user.uid;
                        documentreference=fstore.collection( collectionPath: "users").document(userID);
                        val user1: MutableMap<String, Any> = mutableMapOf()
                        user1.put("Name",fullname);
                        documentreference.set(user1)

                        Log.d(TAG, msg: "Email sent.")
                        Toast.makeText( context: this, text: "Registered successfully. Please check your email for verification.",
                                )
                    }
                    else
                    {
                        Toast.makeText( context: this,task.exception.toString(), Toast.LENGTH_LONG).show();
                    }
                }
            }
        }
}
```

User enters password and confirms:

If valid, creates new Firebase user and sends verification email.

Saves user name in Firebase database if successful.

Show's success/error message based on registration outcome.

#### 4.Animation:

```
var bq=findViewById<ImageView>(R.id.background)
var animation=findViewById<LottieAnimationView>(R.id.task1)
var heading=findViewById<TextView>(R.id.heading)

bq.animate() ViewPropertyAnimator{
    .translationY( value: -2200f) ViewPropertyAnimator
    .setDuration(1000)
    .setStartDelay(4000)
    .start()

animation.animate() ViewPropertyAnimator{
    .translationY( value: 1800f) ViewPropertyAnimator
    .setDuration(1000)
    .setStartDelay(4000)
    .start()

heading.animate() ViewPropertyAnimator{
    .translationY( value: 1400f) ViewPropertyAnimator
    .setDuration(1000)
    .setStartDelay(4000)
    .start()
```

Code animates 3 views (image, animation, text) with different offsets. All views share a 1-second animation duration and 4-second delay. Image moves up, animation moves down, and text moves down further. Demonstrates reusable animation with individual adjustments.

#### 5.Data Class:

```
@data class DataClass(
    var postKey: String?=null,
    var title: String?=null,
    var description: String?=null,
    var date1: String?=null,
    var date: String?=null,
    var month: String?=null,
    var day: String?=null,
    var time: String?=null,
    var status: String?=null,
    var userid: String?=null
){
    constructor() : this(
        postKey: null,
        title: null,
        description: null,
        date1: null
    )

    fun updatePostKey(key: String) {
        this.postKey = key
    }
}
```

Data class: Data Class stores details about a post.  
Attributes: Title, description, date, user ID, and status.  
Constructors: Primary: Set title (optional), key later using method.  
Secondary: All attributes start as null.  
Method: Update post Key after object creation.

## 6.Task deletion and updating in the database:

```
buttonSave.setOnClickListener { it: View!
    val title = editTextTitle.text.toString()
    val description = editTextDescription.text.toString()
    val date1 = editTextDate.text.toString()
    val date = datesdf.toString()
    val month = monthsdf.toString()
    val day = dayOfWeek.toString()
    val time = editTextTime.text.toString()
    if (title.isNotEmpty() && description.isNotEmpty() && date1.isNotEmpty() && time.isNotEmpty()) {
        val currentUser = FirebaseAuth.getInstance().currentUser
        val post = currentUser?.let { user ->
            DataClass(
                postKey: null,
                title,
                description,
                date1,
                date,
                month,
                day,
                time,
                status: "UPCOMING",
                currentUser.vid
            )
        }
        post?.let { addPost(it) }
```

User enters required details (title, description, date, time) for a new post.

System validates that all necessary fields are filled.

A new post object is constructed with entered details and a designated status.

Valid post object is submitted using a dedicated submission function.

## 7.Binding data to the card view:

```
override fun onBindViewHolder(holder: MyViewHolder, position: Int) {
    // binding data to the card view
    Log.d("Adapter", "onBindViewHolder: position $position")
    val item = taskList[position]
    item.title?.let { holder.setTaskTitle(item.title ?: "") }
    item.description?.let { holder.setTaskDescription(item.description ?: "") }
    //item.date1?.let { holder.setTaskDate1(item.date1 ?: "") }
    item.date?.let { holder.setTaskDate(item.date ?: "") }
    item.month?.let { holder.setTaskMonth(item.month ?: "") }
    item.day?.let { holder.setTaskDay(item.day ?: "") }
    item.time?.let { holder.setTaskTime(item.time ?: "") }
    item.status?.let { holder.setTaskStatus(item.status ?: "") }
    holder.options.setOnClickListener { it: View!
        showPopupMenu(holder.options, holder.adapterPosition, holder.status, item.postKey)
    }
}
```

Loop and Bind: Data at position fills card view elements through setter methods.

Options Click: Triggers listener for card's options menu.

Popup Menu: Shows delete complete and update option on click.

Dynamic Interaction: Combines data binding, user interaction, and contextual menus.

## 8.Connecting recycler view and card view using adapter:

```
recyclerView = findViewById(R.id.taskRecycler)
recyclerView.setHasFixedSize(true)
recyclerView.layoutManager = LinearLayoutManager(context: this)
if (adapter == null) {
    list = mutableListOf()
    Toast.makeText(context: this, text: "sending to adapter", Toast.LENGTH_SHORT).show()
    adapter = MyAdapter(context: this, list!!, FirebaseAuth.getInstance())
    recyclerView.adapter = adapter
    Toast.makeText(context: this, text: "after adapter", Toast.LENGTH_SHORT).show()
}
recyclerView.addOnScrollListener(object : RecyclerView.OnScrollListener() {
    override fun onScrolled(recyclerView: RecyclerView, dx: Int, dy: Int) {
        super.onScrolled(recyclerView, dx, dy)
        val isBottom = !recyclerView.canScrollVertically(direction: 1)
        if (isBottom) {
            Toast.makeText(context: this@Home, text: "Reached Bottom", Toast.LENGTH_SHORT).show()
        }
    }
})
adapter!!.fetchAllTasks()
```

Setup recycler view: Find, configure, and set layout manager.

Initialize adapter: Create or reuse an adapter with context and initial data.

Attach adapter: Assign the adapter to the recycler view.

Detect bottom scroll: Monitor scroll events and show message upon reaching the bottom.

## 9.Fetching task details from database and reflecting it on the recycler view:

```
val databaseReference = FirebaseDatabase.getInstance().getReference(path: "tasks")
valueEventListener = databaseReference.addValueEventListener(object : ValueEventListener {
    @SuppressWarnings("NotifyDataSetChanged")
    override fun onDataChange(dataSnapshot: DataSnapshot) {
        list?.clear()

        val currentUserID = FirebaseAuth.getInstance().currentUser?.uid
        for (postSnapshot in dataSnapshot.children) {
            val postId = postSnapshot.key
            val post = postSnapshot.getValue(Data::class.java)?.withId<Data>(postId!!)
            if (post != null && post.userid == currentUserID) {
                list?.add(post)
            }
        }

        Toast.makeText(context: this@Home, text: "on datachange: ${list?.size}", Toast.LENGTH_SHORT).show()
        //Toast.makeText(this@Home, "on datachange: ${list?.get(0)}", Toast.LENGTH_SHORT).show()
        adapter?.notifyDataSetChanged()
        Toast.makeText(context: this@Home, text: "on datachange after", Toast.LENGTH_SHORT).show()
    }

    override fun onCancelled(databaseError: DatabaseError) {
        Toast.makeText(context: this@Home, databaseError.message, Toast.LENGTH_SHORT).show()
    }
})
```

Monitor database: Listens for updates in a specific data path.

Filter and process data: Extracts relevant user-specific data.

Update UI: Informs UI component about changes for data refresh



## 10.Updating data in the database:

```
AlertDialog.Builder(context) AlertDialog.Builder
.setTitle("Update Details") AlertDialog.Builder()
.setView(updateView)
.setPositiveButton(text: "Update") { dialog, which ->
    val updatedTitle = editTextTitle.text.toString()
    val updatedDescription = editTextDescription.text.toString()
    val updatedDate = editTextDate.text.toString()
    val updatedTime = editTextTime.text.toString()
    val item = taskList[position]
    var date = item.date
    var month = item.month
    var day = item.day
    if(flag) {
        date = datesdf.toString()
        month = monthsdf.toString()
        day = dayOfWeek.toString()
    }

    if (updatedTitle.isNotEmpty() && updatedDescription.isNotEmpty() && updatedDate.isNotEmpty() && updatedTime.isNotEmpty()) {
        val currentUser = FirebaseAuth.getInstance().currentUser
        val post = currentUser?.let { user ->
            DataClass(postKey,updatedTitle, updatedDescription, updatedDate,date, month, day,updatedTime,item.status, user.uid)
        }
        post?.let { editPost(it) }
    }
    else
    {
        Toast.makeText(context, text: "Please verify all input fields", Toast.LENGTH_SHORT).show()
    }
    dialog.dismiss()
    updateAlarm(item.updatedTime, updatedDate)
```

Edit Dialog: Displays options to modify post details (title, description, date, time).

Gather Updates: Retrieves user input for changed details.

Validate and Build: Checks required fields and creates updated DataClass object.

Submit or Dismiss: Updates post if valid, shows error otherwise, and closes dialog.

## 11. Scheduling, canceling and updating the alarms:

```
private fun scheduleAlarm(item: Data, context: Context, timeInMillis: Long) {
    val alarmManager = context.getSystemService(Context.ALARM_SERVICE) as AlarmManager
    val alarmIntent = Intent(context, AlarmReceiver::class.java)
    Log.d(tag: "YourAdapter", msg: "entered schedule2")
    alarmIntent.putExtra(name: "message", item.title)
    alarmIntent.action = "android.intent.action.ALARM_RECEIVER"
    val pendingIntent = PendingIntent.getBroadcast(context, item.postKey.hashCode(), alarmIntent, flags PendingIntent.FLAG_UPDATE_CURRENT or PendingIntent.FLAG_IMMUTABLE)
    Log.d(tag: "YourAdapter", msg: "actual time1 ${System.currentTimeMillis()} and time set1 ${timeInMillis}")
    if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.M) {
        alarmManager.setExactAndAllowWhileIdle(AlarmManager.RTC_WAKEUP, timeInMillis, pendingIntent)
    } else {
        alarmManager.setExact(AlarmManager.RTC_WAKEUP, timeInMillis, pendingIntent)
    }
}

private fun cancelAlarm(item: Data, context: Context) {
    val alarmManager = context.getSystemService(Context.ALARM_SERVICE) as AlarmManager
    val alarmIntent = Intent(context, AlarmReceiver::class.java).let { intent ->
        PendingIntent.getBroadcast(context, item.postKey.hashCode(), intent, PendingIntent.FLAG_IMMUTABLE)
    }
    alarmManager.cancel(alarmIntent)
}

private fun updateAlarm(item: Data, newTime: String, newDate: String) {
    val alarmTime = SimpleDateFormat(pattern: "dd/MM/yyyy HH:mm", Locale.getDefault()).parse(source: "$newDate $newTime")?.time ?: 0
    cancelAlarm(item, context)
    scheduleAlarm(item, context, alarmTime)
}
```

**Scheduling Alarms:** Sets up alarms for specific events or reminders, ensuring they trigger even when the device is idle.

**Canceling Alarms:** Dismisses existing alarms to prevent unwanted notifications or actions.

**Updating Alarms:** Modifies scheduled alarm times to reflect changes in event details.

## 12. Creating notification channel:

```
private fun createNotificationChannel(context: Context) {
    if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.O) {
        val name = "Alarm"
        val descriptionText = "Alarm Notification Channel"
        val soundUri = RingtoneManager.getDefaultUri(RingtoneManager.TYPE_NOTIFICATION)
        // val soundUri = Uri.parse("android.resource://${context.packageName}/${R.raw.alarm_sound}")
        val importance = NotificationManager.IMPORTANCE_DEFAULT
        val channel = NotificationChannel(CHANNEL_ID, name, importance).apply {
            this.description = descriptionText
            setSound(soundUri, AudioAttributes.Builder().setUsage(AudioAttributes.USAGE_NOTIFICATION).build())
            enableVibration(vibration: true)
            vibrationPattern = vibrationPattern
        }
        val notificationManager: NotificationManager = context.getSystemService(Context.NOTIFICATION_SERVICE) as NotificationManager
        notificationManager.createNotificationChannel(channel)
        Log.d(tag: "YourAdapter", msg: "entered creation")
    }
}
```

**Alarm Notification Channel:** Sets up a dedicated channel for alarm notifications.

**Channel Settings:** Defines name, description, sound, vibration, and priority level.

**Registration:** Creates and registers the channel with the system's notification manager.

### 13. Alarm receiver notification properties:

```
class AlarmReceiver : BroadcastReceiver(){
    override fun onReceive(context: Context, intent: Intent) {
        val message = intent.getStringExtra( name: "message")
        Log.d( tag: "YourAdapter", msg: "entered reciever")
        createNotificationChannel(context)
        //val soundUri = Uri.parse("android.resource://${context.packageName}/${R.raw.alarm_sound}")
        val notification = NotificationCompat.Builder(context, CHANNEL_ID)
            .setSmallIcon(R.drawable.ic_alarm)
            .setContentTitle("Remainder")
            .setContentText(message)
            .setVibrate(longArrayOf(0, 1000, 1000, 1000))
            .setSound(RingtoneManager.getDefaultUri(RingtoneManager.TYPE_NOTIFICATION))
            //setSound(soundUri)
            .setColor(ContextCompat.getColor(context, R.color.colorAccent))
            .setAutoCancel(true)
            .setPriority(NotificationCompat.PRIORITY_DEFAULT)
            .build()
        Log.d( tag: "YourAdapter", msg: "notification $notification")
        val notificationManager = NotificationManagerCompat.from(context)
        notificationManager.notify(NOTIFICATION_ID, notification)
        Log.d( tag: "YourAdapter", msg: "successful creation")
    }
}
```

Receive Alarm: Handles trigger notification for scheduled alarm.

Build Notification: Creates visual and audible alert with message and vibration.

Display and Log: Shows notification and logs success for monitoring.

### 14. Displaying the task details of a particular date when the date is selected in the calendar:

```
calendarView.setOnDateChangeListener { _, year, month, dayOfMonth ->
    val selectedDate = String.format("%02d/%02d/%d", dayOfMonth, month + 1, year)
    Toast.makeText( context: this@CalendarActivity, text: "Selected date: $selectedDate", Toast.LENGTH_SHORT).show()
    fetchTasksForDate(selectedDate) { tasks: List<Data> ->
        tasksTextView.text = tasks.joinToString( separator: "\n\n") { task ->
            buildString { this: StringBuilder
                append("Title: ${task.title}?: ""\n")
                append("Description: ${task.description}?: ""\n")
                append("Time: ${task.time}?: """)
            }
        }
    }
}
```

Pick date: User selects a date on the calendar.

Fetch tasks: App retrieves tasks matching the chosen date.

Display tasks: List of tasks with title, description, and time shows in a text view.

Individual details: Each task has its own information.

fetchTasksForDate fetches data, tasksTextView is the target text view, and specific functions like joinToString and buildString are used for string formatting.

## RESULTS(SCREENSHOTS):

### 1.Splash screen:

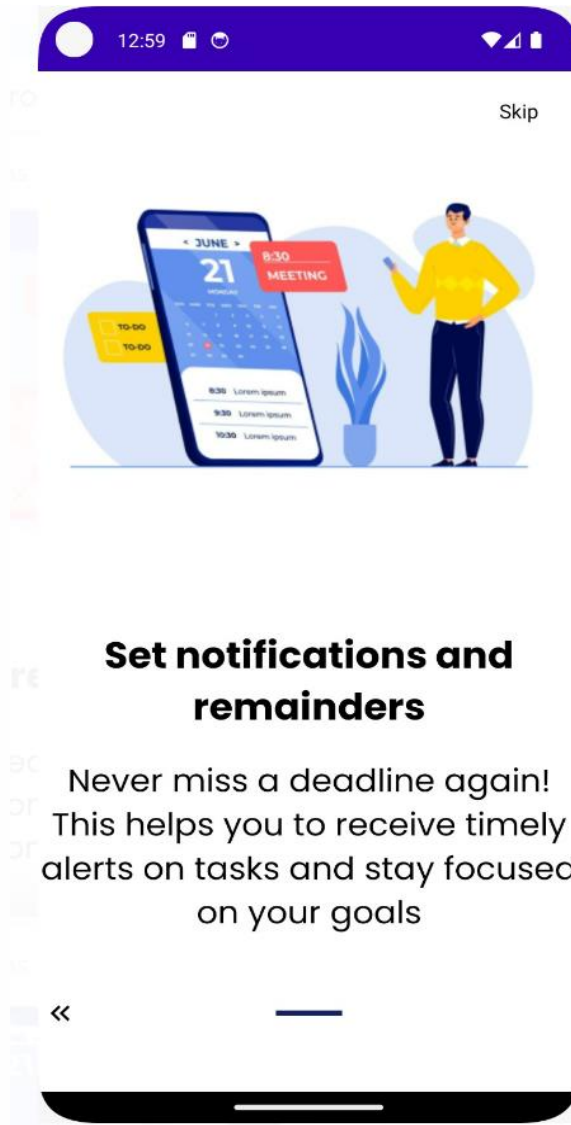
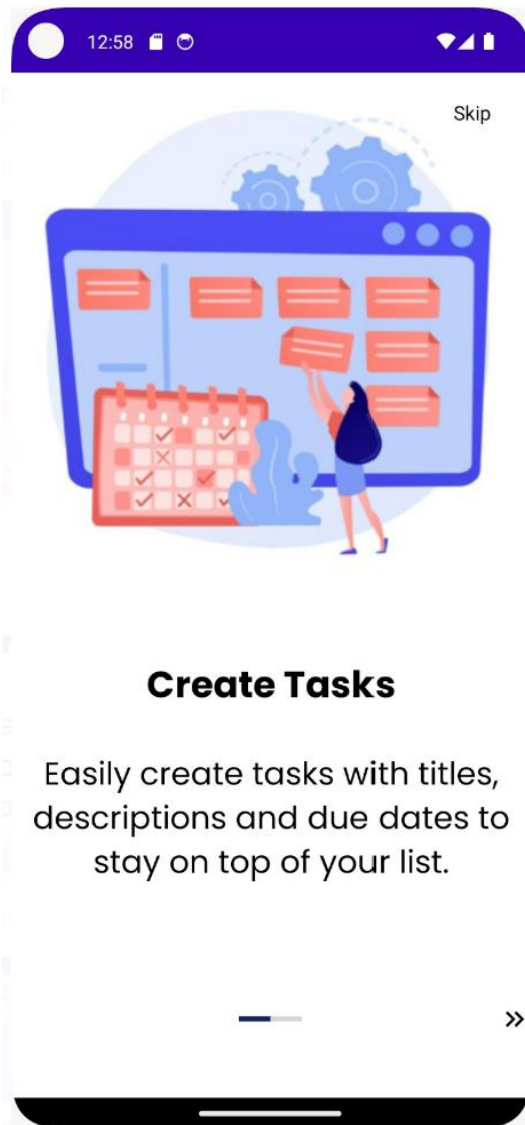


Splashscreen

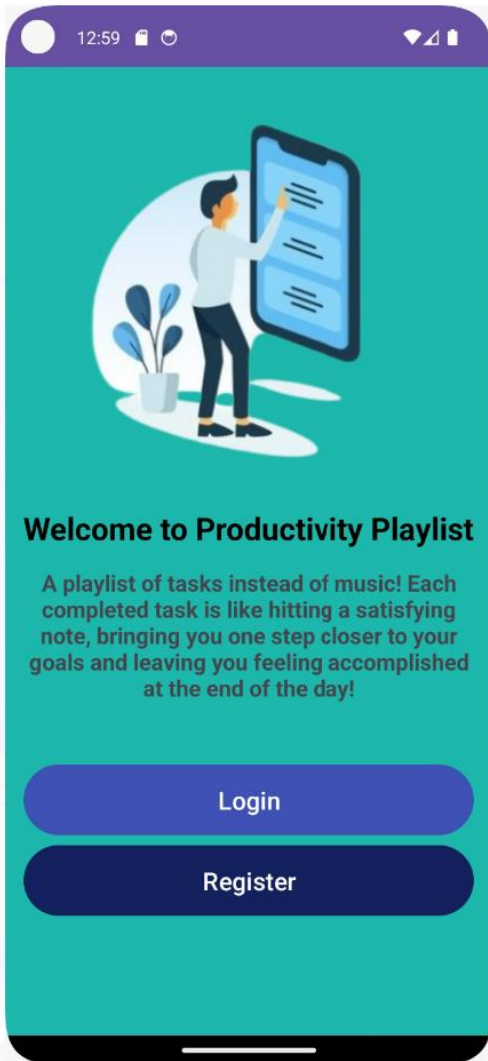
## 2. Animated front screen:



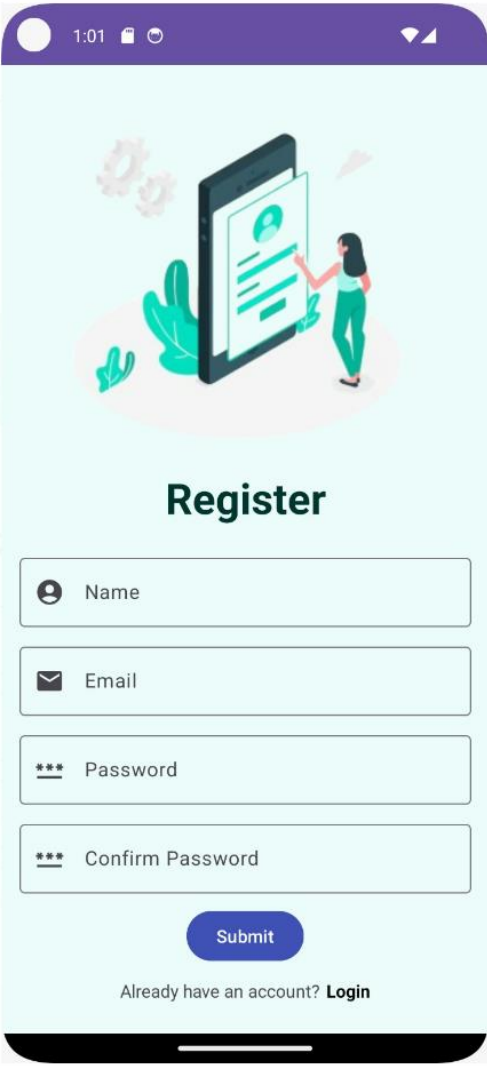
### 3. On boarding pages with progress bar:



#### 4. Welcome page:



## 5.Register:



A mobile application registration screen with a purple header bar showing the time 1:01 and status icons. The main content area has a light blue background with an illustration of a person interacting with a large smartphone. Below the illustration, the word "Register" is displayed in a bold, dark font. There are four input fields: "Name" with a person icon, "Email" with an envelope icon, "Password" with three asterisks, and "Confirm Password" with three asterisks. A blue "Submit" button is centered below the fields. At the bottom, there is a link that says "Already have an account? Login".

1:01

Register

Name

Email

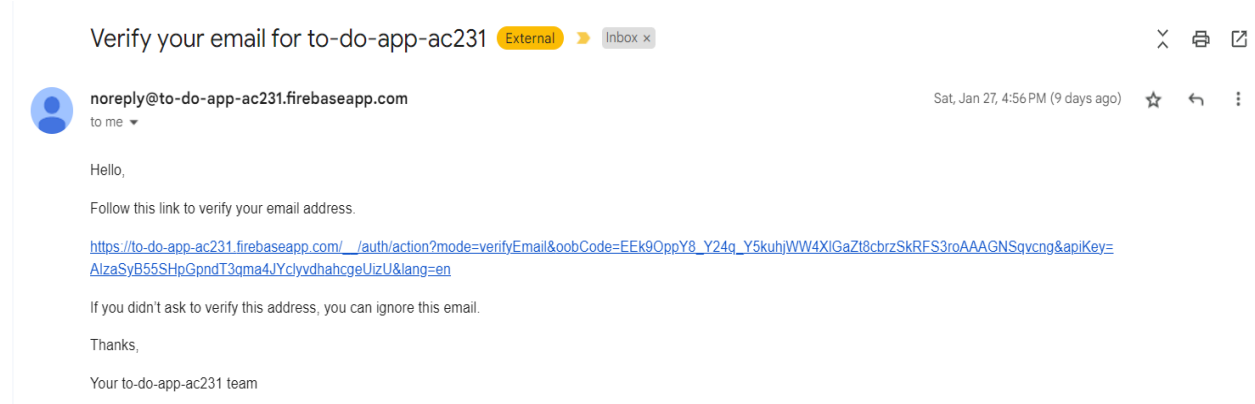
\*\*\* Password

\*\*\* Confirm Password

Submit

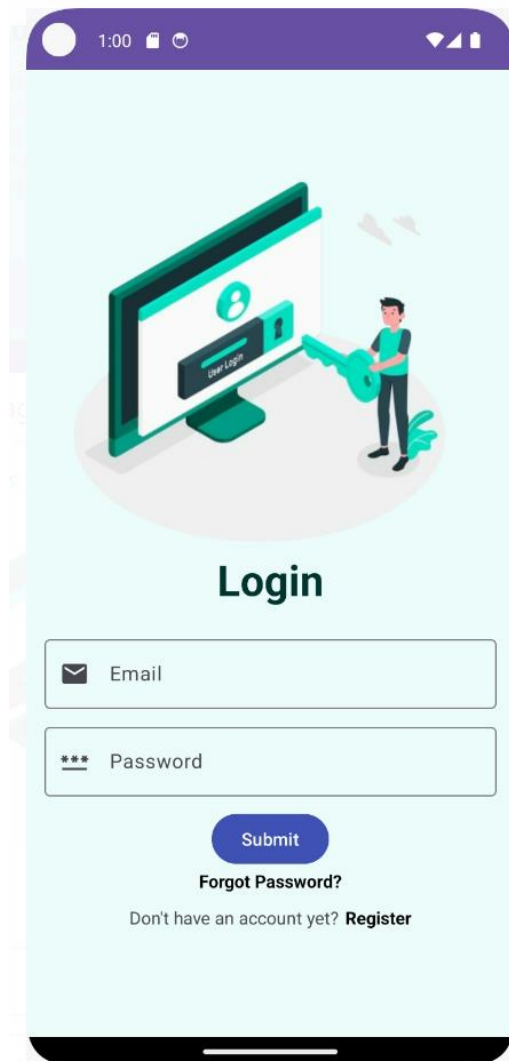
Already have an account? [Login](#)

## Verification Email:





## 6.Login:



A mobile application login screen with a purple header bar. The header contains a white circle icon, the time '1:00', and status icons for battery, signal, and Wi-Fi. The main content area has a light blue background. At the top, there is an illustration of a person holding a large key next to a computer monitor displaying a login form. Below the illustration, the word 'Login' is centered in a bold, dark font. Underneath, there are two input fields: the first is labeled 'Email' with an envelope icon, and the second is labeled 'Password' with three asterisks. A blue 'Submit' button is positioned below the password field. At the bottom, there is a link for 'Forgot Password?' and a text prompt 'Don't have an account yet?' followed by a 'Register' link.

1:00

Login

Email

\*\*\* Password

Submit

Forgot Password?

Don't have an account yet? Register

## 7.Home page:



## 8.Adding task:

The screenshot shows a mobile application interface for adding a task. At the top, there is a status bar with the time 7:48:01, network speed 4.00 Kbps, and battery level 47%. Below the status bar is a navigation bar with a back arrow and the title "Add a task". The main content area contains four input fields: "Task title" with the value "MAD project", "Task description" with the value "Project Review", "Task date" with the value "13/02/2024", and "Task time" with the value "19:45". A blue button labeled "ADD TASK" is positioned below the input fields. At the bottom of the screen is a black navigation bar with three icons: a hamburger menu, a square, and a triangle.

7:48:01 4.00 Kbps 47%

← Add a task

Task title  
MAD project

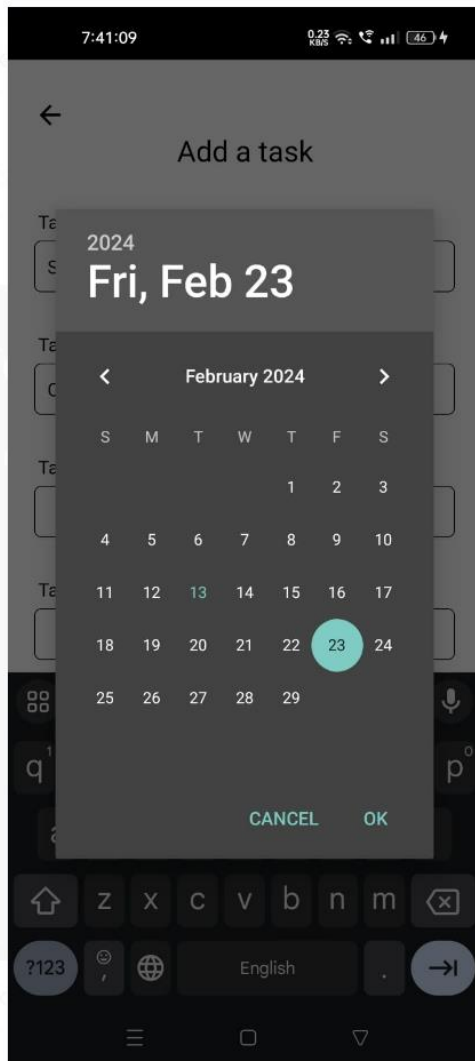
Task description  
Project Review

Task date  
13/02/2024

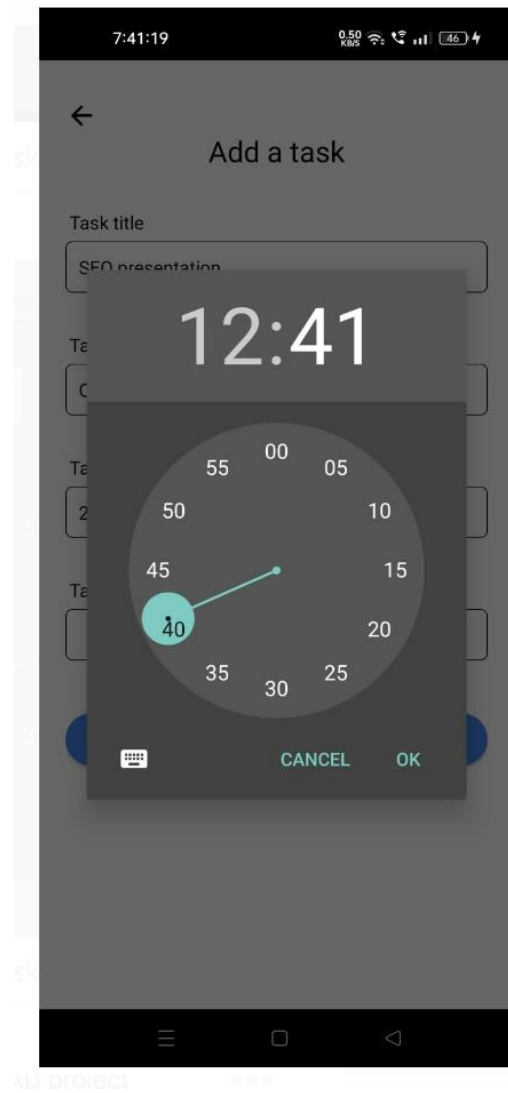
Task time  
19:45

ADD TASK

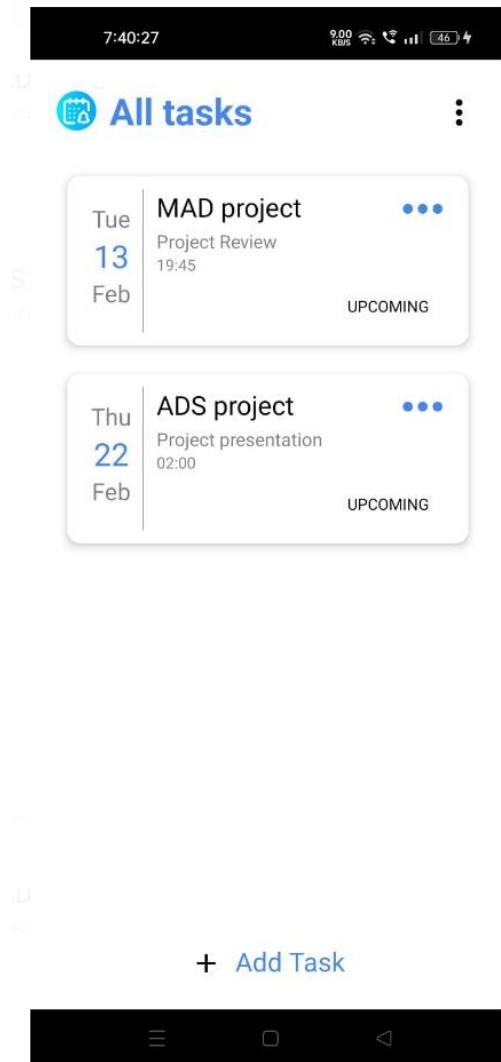
## 9.Task date selection:



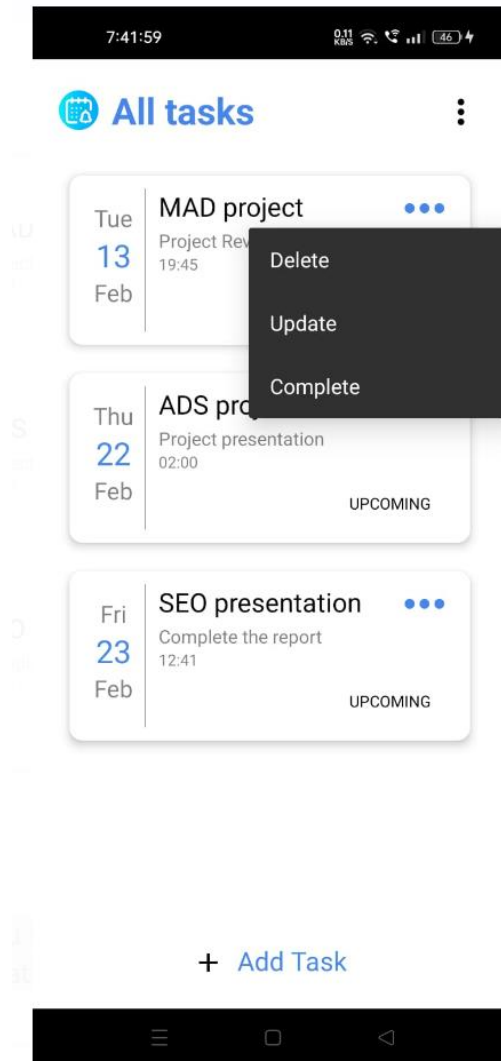
## 10.Task time selection:



## 11.Adding task:



## 12. Popup menu for delete, update and changing status to completed:



### 13.Updating Details:

7:42:12 0.30 KB/s 46%

 All tasks

Update Details

← Update the task

Task title

MAD project

Task description

Project Review

Task date

13/02/2024

Task time

19:45

CANCEL UPDATE

+ Add Task

7:42:42 470 KB/s 47%

 All tasks

Wed 14 Feb MAD project Project Review 19:45 UPCOMING

Thu 22 Feb ADS project Project presentation 02:00 UPCOMING

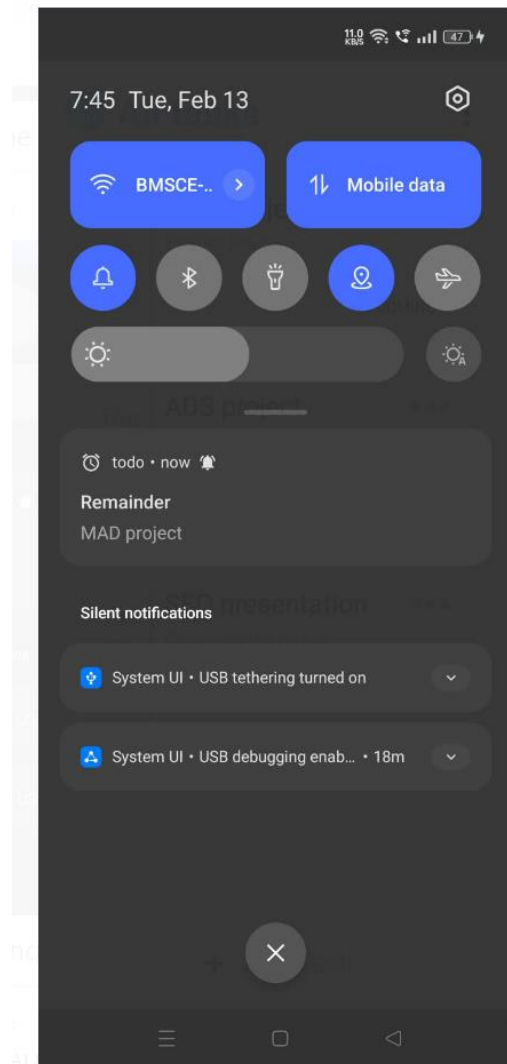
Fri 23 Feb SEO presentation Complete the report 12:41 UPCOMING

 Task updated successfully

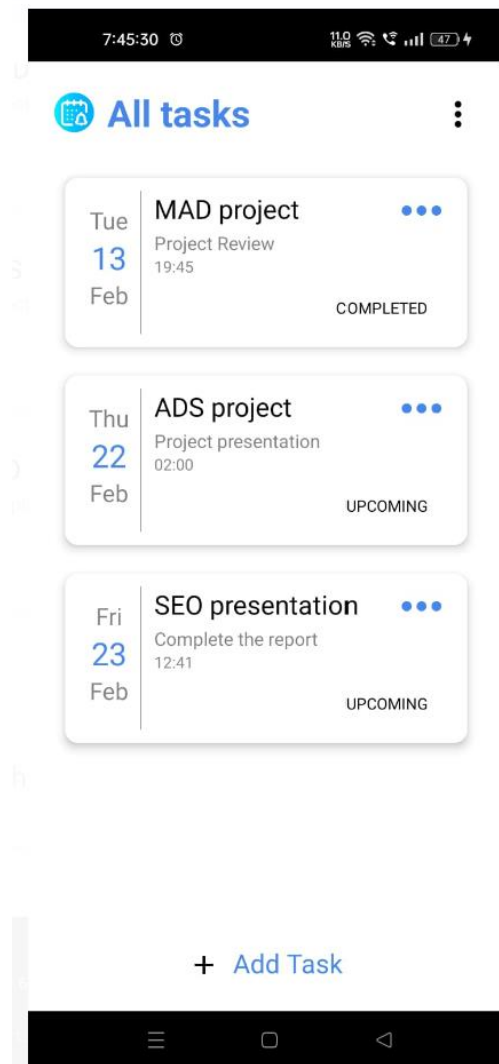
+ Add Task



## 14. Remainder notification:



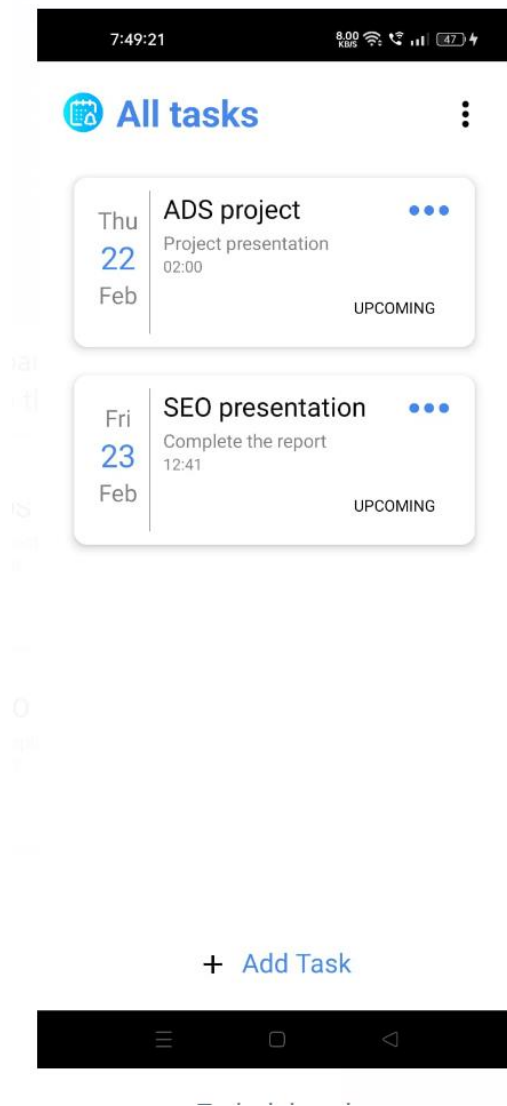
## 15.Task status changed to complete:



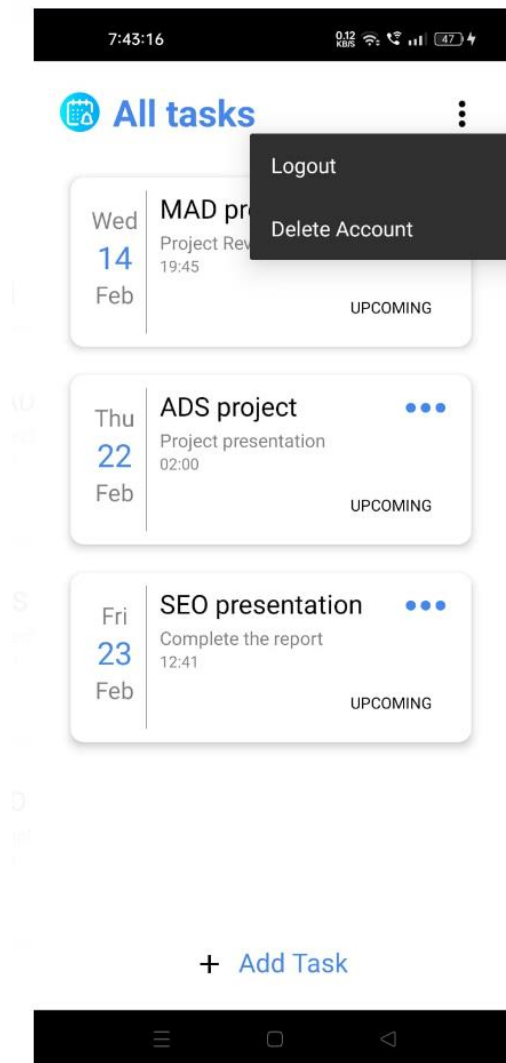
## 16.Tasks on a particular date are displayed in the calendar:



## 17.Task deleted:



## 18. Popup menu for logout and delete account:



## **CONCLUSION AND FUTURE ENHANCEMENT:**

The development of the To-Do List Application represents a significant step forward in addressing the challenges of modern task management. Through intuitive design, robust functionality, and seamless integration of features such as alarms, notifications, and authentication, the application offers users a comprehensive solution for organizing their tasks and staying on top of their commitments.

By leveraging technology to streamline task management processes, the Application empowers users to enhance productivity, reduce stress, and achieve greater efficiency in their daily lives. With personalized reminders, and secure authentication mechanisms, the application caters to the diverse needs and preferences of users, ensuring a tailored and user-centric experience.

The Application represents not only a valuable tool for individuals seeking to improve their task management skills but also a testament to the transformative power of technology in empowering individuals to lead more organized, productive, and fulfilling lives. As the application continues to evolve and expand its features, it holds the promise of further enhancing the way users manage their tasks and achieve their goals in an increasingly fast-paced and dynamic world.

This task scheduling app can be further enhanced by empowering collaboration. Users could manage shared projects, assign roles, and receive real-time updates. Advanced features like task dependencies, recurring tasks, and tagging would improve organization. By prioritizing user needs and market trends, these enhancements can attract a wider audience and transform the app from a personal organizer to a collaborative hub.

## REFERENCES:

[https://youtu.be/\\_qmU3tUQTBk?si=cY3aYVPK9uPD4\\_ia](https://youtu.be/_qmU3tUQTBk?si=cY3aYVPK9uPD4_ia)  
[https://youtube.com/playlist?list=PLhhNsarqV6MQrlXyjuaLJf\\_\\_takk51\\_d&si=IWyXxCoK9qmS5-Ma](https://youtube.com/playlist?list=PLhhNsarqV6MQrlXyjuaLJf__takk51_d&si=IWyXxCoK9qmS5-Ma)  
<https://firebase.google.com/docs/auth>  
<https://developer.android.com/reference>  
<https://developers.google.com/android/guides/google-services-plugin>  
<https://developer.android.com/develop/background-work/services/alarms/schedule>